

C++ Arrays and STL Containers Overview

Question: Are arrays containers?

Yes. Arrays are containers because they store multiple elements of the same type. In C++, built-in arrays act as containers, while `std::array` is an official Standard Library container.

Types of arrays in C++

Built-in (C-style) array: Fixed-size array built into the language.

Multidimensional array: Arrays of arrays, e.g. `int matrix[2][3]`.

Dynamic array (`new[]`): Allocated at runtime on the heap; manual `delete[]` required.

`std::array`: Fixed-size STL container.

`std::vector`: Dynamic array that automatically resizes.

STL Types Explained

`std::vector`: Dynamic array container.

`std::pair`: Utility object holding exactly two values; not a container.

`std::deque`: Double-ended queue supporting efficient insertion/removal at both ends.

`std::list`: Doubly linked list.

`std::map`: Sorted key-value associative container (typically Red-Black Tree).

`std::unordered_map`: Hash-table-based key-value associative container.

`std::set`: Sorted container of unique values.

Which are array-based?

Built-in array: Yes

`std::array`: Yes

`std::vector`: Yes (dynamic array)

`std::deque`: No (segmented storage)

`std::list`: No (linked list)

`std::map`: No (balanced tree)

`std::unordered_map`: No (hash table)

`std::set`: No (balanced tree)

`std::pair`: No (utility type, not a container)